

GUERRA MUNDIAL J

Multithreaded Humans vs Zombies Simulation

Final Project Report
Concurrent & Parallel Programming

Python 3.x | threading | Tkinter | SQLite

1. Executive Overview

Guerra Mundial J is a real-time, multithreaded simulation of a zombie outbreak spreading across a map of the continental United States. Built in Python, it uses the threading module for concurrency, Tkinter for visualization, and SQLite for persistent data storage.

The core concept: every agent in the simulation (human or zombie) runs as an independent thread, making autonomous decisions in parallel. With a default configuration of 300 humans and 5 zombies, the simulation manages approximately 310 concurrent daemon threads, each with its own behavior loop, movement speed, and decision-making logic.

The simulation explores a central research question: **which presidential strategy maximizes long-term human survival?** Through stochastic encounters across thousands of parallel runs, we use batch simulations and SQL-based statistical analysis to compare strategies and identify how parameters like infection probability affect outcomes.

1.1 Technologies

Technology	Purpose
Python 3.x	Core language. threading module for concurrency, abc module for abstract classes.
threading	Thread, Lock, and Event classes for parallel agent execution and synchronization.
queue.Queue	Thread-safe queue for the producer-consumer database write pattern.
Tkinter	Built-in GUI framework for real-time 250x250 grid visualization.
SQLite	Embedded database for persisting simulation results and events. WAL mode enabled.

1.2 Key Numbers

Metric	Value
Grid size	250 x 250 cells (62,500 cells)
Default population	300 humans + 5 zombies
Concurrent threads	~310 (305 agents + 5 background services)
Locks (threading.Lock)	7 (grid, event, ID, result, EventBus, CommandHistory, DB)
Signals (threading.Event)	4 (game_over, antidote_ready, national_alert, pause_event)
Design patterns	7 GoF patterns + producer-consumer concurrency pattern
Unit tests	52 (all passing)
Python files	~15 across 4 packages

2. Description of the Real-World Scenario

The simulation models a zombie pandemic spreading across the continental United States. It explores how autonomous agents with different roles, abilities, and strategies survive (or fail to survive) a rapidly spreading infection through probabilistic encounters.

2.1 The Map

The world is a 250x250 grid representing the continental US. A polygon of 130+ coordinate points defines the coastline, and a ray-casting point-in-polygon algorithm determines which cells are land and which are ocean. Agents can only move on land cells. Four key locations are placed at geographically accurate positions:

Location	Real-World Reference	Function
Outbreak Origin	San Diego, CA (12, 185)	Where the first zombie (José) spawns
Laboratory	CDC Atlanta, GA (170, 155)	Scientists research the antidote here
White House	Washington D.C. (200, 100)	Politicians activate the national strategy
Military Base	Fort Bragg, NC (190, 138)	Safe zone: regenerates food/water, resupplies ammo

2.2 Agent Roles

Humans are divided into four specialized roles, each with unique abilities and goals:

Role	% of Pop.	Special Abilities
Normal Citizen	65%	Basic civilian. No special abilities. Relies on strategy and numbers to survive.
Military	20%	Starts with +15 force bonus and ammo. +25% chance to kill zombies in combat.
Scientist	10%	Can research the antidote at the CDC lab. Needs 3000 ticks near the lab to complete.
Politician	5%	Can reach the White House to issue a national alert, activating the coordination strategy.

2.3 Simulation Flow

The simulation follows a five-phase narrative:

Phase 1 - Outbreak: Zombies appear in San Diego. Humans are unaware.

Phase 2 - Panic: Humans detect nearby zombies and begin fleeing. No coordination yet.

Phase 3 - Alert: A politician reaches the White House and sends a national alert.

Phase 4 - Response: After a bureaucratic delay, the chosen strategy activates. All humans coordinate.

Phase 5 - The End: Either all zombies are eliminated / antidote completed (humans win), or all humans die (zombies win).

2.4 Survival Mechanics

Every agent has food (100) and water (100) that decay each tick (food: -0.15/tick, water: -0.20/tick). When either reaches zero, the agent loses force. If force drops below 5, the agent dies of starvation or dehydration. Agents can recover at the military base refuge (regenerates 2.0 food and water per tick), but are evicted after 150 ticks with a 200-tick cooldown to prevent camping.

2.5 Win Conditions

Humans win if all zombies are eliminated through combat, or if a scientist completes the antidote (3000 research ticks near the CDC lab). **Zombies win** if all humans die from combat, infection, or starvation/dehydration. The WinChecker background thread evaluates these conditions every second.

3. Object-Oriented Structure

3.1 Class Hierarchy

The agent hierarchy follows a two-level inheritance tree rooted in the abstract Agent class:

```
threading.Thread + ABC
|
Agent
/ \
Zombie Human
/ | \ \
Normal Scientist Military Politician
```

Agent inherits from both **threading.Thread** (so every agent runs as its own independent thread) and **ABC** (Abstract Base Class, which forces subclasses to implement `update()` and `get_color()`). Agent cannot be instantiated directly.

Agent defines all common attributes shared by every agent: position on the grid, force (0-100), age, state (calm/running/fighting/infected/dead), a unique thread-safe ID, and the `run()` lifecycle loop. It also provides `clone()` for the Prototype pattern and `distance_to()` for movement calculations.

Class	Inherits From	Adds
Agent (abstract)	threading.Thread, ABC	pos, force, age, state, ID, run(), clone(), die()
Human	Agent	food, water, infect(), survival decay, strategy-based movement
Zombie	Agent	Hunting logic, infection spreading, increased vision range
Normal	Human	Basic civilian behavior (flee, seek refuge)
Scientist	Human	research_counter, antidote research at CDC lab
Military	Human	ammo, force bonus (+15), use_ammo(), base guarding
Politician	Human	influence (60-100), White House navigation, national alert

3.2 Project Structure

File / Package	Responsibility
main.py	CLI entry point. Parses arguments, launches UI or batch mode.
config.py	All simulation constants: grid size, probabilities, timing, colors, locations.
signals.py	4 global threading.Event objects for cross-thread signaling.
event_bus.py	Observer pattern: thread-safe publish-subscribe event system.

File / Package	Responsibility
commands.py	Command pattern: 7 command classes + CommandHistory for replay.
agents/base_agent.py	Abstract Agent base class with Template Method lifecycle.
agents/human.py	Human + 4 role subclasses (Normal, Scientist, Military, Politician).
agents/zombie.py	Zombie agent with hunting and infection logic.
simulation/engine.py	Central orchestrator: creates world, agents, manages background threads.
simulation/world.py	Thread-safe 250x250 grid with land mask, event queue, snapshots.
simulation/combat.py	Probabilistic combat resolution using force ratio, age, and role.
simulation/movement.py	Strategy pattern: flee, group, military_first, random behaviors.
db/database.py	Singleton SQLite layer with async writer thread (producer-consumer).
db/stats.py	SQL-based statistical analysis (strategy comparison, sensitivity).
db/models.py	SQL schema definitions and query templates.
ui/app.py	Main Tkinter application with snapshot-render-display pipeline.
ui/grid_canvas.py	Pre-allocated oval pool for efficient grid rendering (62,500 ovals).
tests/test_db.py	52 unit tests for the database and agent layers.

4. Parallelism and Thread Architecture

The simulation is fundamentally parallel: every agent is an autonomous thread making independent decisions. This section details how we manage approximately 310 concurrent threads safely and efficiently.

4.1 Agent Threads (305 threads)

Every agent inherits from `threading.Thread` and runs as its own daemon thread. The Agent base class defines the `run()` lifecycle loop using the **Template Method** pattern:

```
def run(self):
    while not game_over.is_set() and self._alive:
        pause_event.wait(timeout=0.5)
        if self.state == 'dead': break
        self.update() # subclass-specific behavior
        time.sleep(self.move_delay)
```

Each agent has a different `move_delay` calculated from its age and force. The formula: `delay = TICK_SPEED (0.1s) * age_factor * force_multiplier`. Age factor increases +2% per year over 60. Force multiplier ranges from 1.0 (force=0) to 0.6 (force=100). Result: a young strong soldier moves 2-3x faster than an elderly civilian (minimum delay: 0.04s).

All agent threads are **daemon threads** (`daemon=True`). This means Python kills them automatically when the main thread exits, providing instant cleanup without joining 310 threads. The main thread (Tkinter UI) is the only non-daemon thread.

4.2 Background Service Threads (5 threads)

Thread	Interval	Function
TickCounter	0.1s	Increments the global simulation clock (<code>world.tick</code>).
WinChecker	1.0s	Evaluates win/loss conditions. Sets <code>game_over</code> when met.
InfectionMonitor	0.3s	Scans infected humans. Converts to zombie after 25-tick incubation using Prototype.
StrategyMonitor	continuous	Waits for <code>national_alert</code> , then activates strategy after bureaucratic delay.
DatabaseWriter	on demand	Consumer in producer-consumer pattern. Batches up to 50 events per SQLite transaction.

4.3 Synchronization: 7 Locks

`threading.Lock` objects provide mutual exclusion for shared resources. A lock ensures only one thread accesses the protected resource at a time. We use 7 locks:

Lock	Location	Protects
Grid lock	world.py	The 250x250 grid. Most critical: all 310 threads read/write positions.
Event lock	world.py	The event queue that feeds the UI event log panel.
ID lock	base_agent.py	The global agent ID counter for unique assignment.
Result lock	engine.py	The simulation result variable (humans_win / zombies_win).
EventBus lock	event_bus.py	The subscriber dictionary in the Observer pattern.
CommandHistory lock	commands.py	The combat command log list (chronological record).
Database lock	database.py	The SQLite connection for read operations.

Deadlock avoidance: We follow a strict rule: never hold two locks simultaneously. Each critical section is kept as short as possible. Pathfinding is calculated before acquiring the grid lock; only the brief position swap (remove old, place new) happens inside the lock.

4.4 Synchronization: 4 Signals (threading.Event)

threading.Event objects are boolean flags that any thread can wait on or check. Unlike locks (mutual exclusion), events are used for signaling:

Signal	Initial State	Purpose
game_over	clear (False)	When set, all 310 agent threads exit their run() loops.
antidote_ready	clear (False)	Set when a scientist completes the cure after 3000 research ticks.
national_alert	clear (False)	Set when a politician reaches the White House.
pause_event	set (True)	Controls pause/resume. Cleared = all agents block on wait().

Note: pause_event starts as **set** (running), not clear. When cleared, all agents block on pause_event.wait(timeout=0.5). The timeout ensures agents periodically recheck game_over even while paused.

5. Design Patterns

The project implements seven GoF (Gang of Four) design patterns plus one concurrency pattern. Each solves a specific architectural problem in the multithreaded simulation:

5.1 Template Method (`base_agent.py`)

Problem: The lifecycle loop (check `game_over`, pause, update, sleep) is identical for all agents, but the behavior logic differs completely between humans and zombies.

Solution: `Agent.run()` defines the fixed skeleton algorithm. The `update()` method is declared abstract with `@abstractmethod`, forcing each subclass to provide its own implementation. Human subclasses flee, fight, research, or seek the White House. Zombie implements hunting logic.

Benefit: Adding a new agent type only requires implementing `update()` and `get_color()`. The lifecycle (start, loop, die) is inherited automatically.

5.2 Observer (`event_bus.py`)

Problem: When a zombie dies in combat, the UI needs to update its counter, the engine needs to check win conditions, and stats need recording. Without Observer, `combat.py` would need to import and call each module directly, creating tight coupling.

Solution: The EventBus is a publish-subscribe system with a dictionary mapping event topics (strings) to lists of callback functions. Modules subscribe to events at startup. When combat publishes an event, the EventBus calls all registered callbacks. The publisher has zero knowledge of who is listening.

Benefit: Complete decoupling. Adding a new subscriber (e.g., a sound module) requires one line. `Combat.py` is never modified.

5.3 Command (`commands.py`)

Problem: Combat outcomes need to be both applied to agents AND logged for analytics/replay. Calling `zombie.die()` directly leaves no record.

Solution: Each combat outcome is wrapped in a command object (`EscapeCommand`, `InfectCommand`, `KillHumanCommand`, `KillZombieCommand`). The `CommandHistory` executes the command and stores it in a chronological list. UI actions (Start, Pause, Reset) are also encapsulated as commands.

Benefit: Separation of deciding (probability calculation) from doing (command execution). The history provides a complete log for future replay or post-game analytics.

5.4 Singleton (`db/database.py`)

Problem: 310 threads need database access. Multiple SQLite connections would cause 'database is locked' errors.

Solution: A metaclass (`_DatabaseMeta`) intercepts the `Database()` constructor. The first call creates the instance; every subsequent call returns the same instance. All threads share one connection and one write queue.

Benefit: Thread safety by construction. Impossible to accidentally create a second database connection.

5.5 Prototype (`base_agent.py`)

Problem: When an infected human turns into a zombie, constructing the new `Zombie` object inside the grid lock causes contention because object creation is relatively slow.

Solution: `agent.clone(cls=Zombie, force=...)` creates the new zombie outside the lock, copying position, age, and world reference. Only the brief grid swap (remove human, place zombie) happens under the lock.

Benefit: Minimized lock hold time. Construction (slow) is free; only the grid mutation (fast) needs synchronization.

5.6 Factory Method (`engine.py`)

Problem: Creating 305 agents involves complex logic: role distribution (10% scientists, 20% military, 5% politicians), random attributes, finding free cells, and grid placement.

Solution: `_create_agents()` builds a list of (`AgentClass`, `kwargs`) tuples and iterates through them with a single loop. The engine's `start()` method simply calls this one factory method.

Benefit: Clean separation. Adding a new role means adding tuples to the list, nothing else changes.

5.7 Strategy (`movement.py`)

Problem: Humans need different movement behaviors depending on the active strategy, and the strategy can change mid-simulation when a politician reaches the White House.

Solution: `world.strategy` starts as 'none' (no coordination). When the national alert fires, it changes to the chosen strategy (flee, group, military_first, random). Each tick, humans read `world.strategy` and execute the corresponding movement logic.

Benefit: Behavior changes at runtime without restarting or notifying agents. They simply read the shared variable on their next tick.

5.8 Producer-Consumer (`db/database.py`)

Problem: 310 threads generate combat events. Synchronous writes would block agents and cause SQLite locking errors.

Solution: Agent threads (producers) put events into a queue. Queue without blocking. The DB-Writer thread (consumer) pulls events and writes them to SQLite in batches of up to 50 per transaction.

Benefit: Agent threads never wait for disk I/O. Batched writes are dramatically faster than individual commits. Note: this is a concurrency pattern, not a GoF design pattern.

6. Combat System

When a human and a zombie occupy the same or adjacent cell, the engine calls `resolve_encounter()` in `combat.py`. The combat system operates in three phases:

6.1 Probability Calculation

The function `_calculate_probabilities()` computes raw probabilities for four outcomes based on three factors:

Factor	Effect on Probabilities
Force ratio (<code>human.force / zombie.force</code>)	Higher ratio increases escape chance, decreases infection probability.
Age (>60 years)	Humans over 60 get +8% infection probability (more physically vulnerable).
Role	Military with ammo: +25% zombie kill chance. Without ammo: +5%. Civilians: -2% penalty.

6.2 Outcome Selection

The four probabilities are normalized to sum to 1.0. A `random.random()` value (0 to 1) is generated and mapped proportionally to the four outcomes, creating a weighted random selection. Base probabilities from config: `P_ESCAPE=0.35`, `P_INFECT=0.25`, `P_KILL_ZOMBIE=0.08`.

6.3 Four Possible Outcomes

Outcome	Effect	Applied Via
escape	Human flees. State set to 'running'.	EscapeCommand
human_infected	25-tick incubation, then InfectionMonitor converts to zombie.	InfectCommand
human_dies	Human dies instantly. Removed from the grid.	KillHumanCommand
zombie_dies	Zombie eliminated. Military with ammo has highest chance.	KillZombieCommand

All outcomes are applied through the **Command pattern**: each creates a command object that executes the effect, logs it to `CommandHistory`, and publishes it to the `EventBus` for the Observer pattern.

7. Optimizations and Trade-offs

7.1 Visualization Pipeline

Tkinter is not thread-safe. If agent threads modified the canvas directly, the application would crash. We solved this with a three-stage pipeline:

- 1. Snapshot:** Every 80ms, the main thread acquires the grid lock and takes a fast copy of the current state. Agent threads are blocked only during this brief copy operation.
- 2. Render:** Pre-allocated ovals on the canvas are updated. Instead of creating and destroying 62,500 shapes every frame, all ovals are created once at startup and only their color is changed with `itemconfig()`. This reduces rendering from $O(n)$ create/destroy to $O(\text{changed})$ updates.
- 3. Display:** All UI updates happen on Tkinter's main thread via `self.after(80, callback)`, which is the only thread-safe way to schedule work on the main thread.

7.2 Database Optimization

Batched writes: The DB-Writer accumulates up to 50 events (`BATCH_SIZE`) and writes them in a single SQLite transaction, reducing disk syncs by up to 50x.

WAL mode: Write-Ahead Logging is enabled for concurrent reads during writes, ideal for the multi-threaded workload.

7.3 Lock Contention Minimization

Short critical sections: Pathfinding and probability calculations happen outside locks. Only the final grid mutation occurs inside.

Prototype pattern: New Zombie objects are constructed outside the grid lock. Only the brief remove + place swap requires the lock.

EventBus copy-then-iterate: `publish()` copies the listener list inside the lock, then calls callbacks outside. Long callbacks don't block other publishers.

Euclidean distance: Used over complex pathfinding algorithms for movement calculations, providing sufficient accuracy at much lower computational cost.

Land mask caching: The US polygon land mask is computed once at world creation and cached, avoiding expensive ray-casting on every movement check.

7.4 GIL Trade-off

Python's Global Interpreter Lock (GIL) means only one thread executes Python bytecode at a time. However, `time.sleep()` releases the GIL, so while one agent sleeps (most of its time), others can execute. The simulation is I/O-bound (`sleep` + `rendering`), not CPU-bound, making threads

appropriate despite the GIL. True CPU parallelism via multiprocessing would require serializing agent state across processes, adding significant complexity for minimal benefit.

8. Challenges Faced

8.1 Race Conditions on the Grid

With 310 threads reading and writing the same 250x250 grid, race conditions were inevitable without proper synchronization. Two agents could move to the same cell simultaneously, causing one to 'disappear'. The grid lock solved this, but we had to keep critical sections minimal to avoid turning the simulation into a sequential bottleneck.

8.2 Circular Imports

The Human and Zombie classes reference each other (humans detect zombies, zombies hunt humans). Direct imports would create circular dependencies. We solved this using `agent.__class__.__name__` for string-based type checking instead of `isinstance()`, and `TYPE_CHECKING` guards for type hints that are only evaluated by static analyzers, not at runtime.

8.3 UI Thread Safety

Tkinter crashes if modified from any thread other than the main thread. We discovered this through runtime errors and solved it with the snapshot-render-display pipeline. All rendering uses `self.after()` to schedule updates on the main thread. This was the most time-consuming debugging challenge.

8.4 SQLite Single-Thread Limitation

SQLite does not handle concurrent writes, producing 'database is locked' errors. The producer-consumer pattern with a dedicated writer thread eliminated this completely. Events are queued and written in batches, decoupling simulation speed from disk I/O.

8.5 Zombie Conversion Performance

Initially, converting an infected human into a zombie (create object + remove old + place new) happened entirely under the grid lock, creating noticeable lag. The InfectionMonitor now performs an atomic grid swap using the Prototype pattern: the new Zombie is constructed outside the lock, and only the brief swap operation requires synchronization.

9. Test Results and Performance Metrics

9.1 Unit Tests

52 unit tests verify the database and agent layers. Tests use in-memory SQLite databases (':memory:') for isolation and speed. All tests pass consistently.

Test Suite	Tests	Coverage
Agent tests	21	Agent creation, state transitions, movement, combat interactions.
Engine tests	12	Simulation lifecycle, win conditions, thread management.
Database tests	17	Save/load simulations, events, result updates, stats queries.
Snapshot tests	2	Grid snapshot consistency under concurrent modification.

Result: **52/52 passing**. Run with: `python -m pytest tests/`

9.2 Batch Simulation Results

We ran 89 batch simulations across all strategies and multiple infection probabilities to compare outcomes. Results are computed using SQL aggregation queries on the simulations table.

9.2.1 Win Rate by Strategy

The `analyze_strategies()` query groups simulations by strategy and calculates the human win rate:

```
SELECT strategy, COUNT(*) AS total,  
SUM(CASE WHEN result = 'humans_win' THEN 1 ELSE 0 END) AS human_wins,  
ROUND(100.0 * human_wins / total, 2) AS win_rate_pct  
FROM simulations GROUP BY strategy
```

Results from 89 batch simulations:

Strategy	Total Sims	Human Wins	Win Rate
none	43	43	100.0%
random	10	7	70.0%
flee	13	5	38.5%
military_first	8	3	37.5%
group	15	0	0.0%

Key findings: The 'none' strategy (no coordination) achieved 100% win rate across 43 simulations. Without centralized movement, humans spread naturally across the map, making it harder for zombies to infect clusters. The 'group' strategy performed worst (0%) because clustering creates dense groups where a single infection cascades rapidly. The 'random' strategy (70%) outperformed

organized strategies ('flee' at 38.5%, 'military_first' at 37.5%), suggesting unpredictable movement is harder for zombies to counter than organized but predictable patterns.

9.2.2 Sensitivity to Infection Probability (p_infect)

The sensitivity_analysis() query groups simulations by p_infect and calculates win rates per bucket:

```
SELECT p_infect, COUNT(*) AS total,  
ROUND(100.0 * SUM(CASE WHEN result = 'humans_win' THEN 1 ELSE 0 END)  
/ COUNT(*), 2) AS win_rate_pct  
FROM simulations GROUP BY p_infect ORDER BY p_infect
```

p_infect	Total Sims	Win Rate	Trend
0.2	5	100.0%	Humans dominate
0.3	8	87.5%	Humans favored
0.4	73	61.6%	Balanced
0.6	3	33.3%	Zombies favored

Analysis: Clear inverse correlation between infection probability and human survival. At p_infect=0.2, humans win every simulation because infections are rare enough for combat and the antidote to outpace the spread. At p_infect=0.4 (our primary testing value, 73 simulations), outcomes are balanced at 61.6%, producing the most varied and interesting simulations. At p_infect=0.6, zombies win two-thirds of the time as infections spread faster than humans can respond.

10. Lessons Learned

Design for concurrency from the start. Synchronization cannot be bolted on after the fact. Identifying shared resources and placing locks correctly must happen during initial architectural design, not as debugging afterthoughts.

Minimize lock hold times. The key to performance in a multithreaded system is keeping critical sections as short as possible. Calculate outside the lock, mutate inside. This principle guided our use of the Prototype pattern and the snapshot pipeline.

Design patterns reduce coupling. Observer and Command eliminated direct dependencies between combat, UI, and engine modules. This made debugging significantly easier because changes to one module did not cascade through the entire codebase.

Producer-consumer is essential for asymmetric I/O. Whenever many threads produce data and one system consumes it (database, file logger), a queue-based approach prevents bottlenecks and thread contention.

Test what you can, simulate what you can't. Unit tests cover the deterministic database layer (52 tests). The stochastic simulation itself is validated through batch runs and statistical analysis rather than traditional assertions.

The GIL is not a dealbreaker. Python's GIL prevents true CPU parallelism, but for I/O-bound workloads (agents spending most time in `sleep()`), threading provides effective concurrency. The architectural simplicity of one-thread-per-agent outweighs the theoretical CPU limitation.